

Het integreren van Python modules in een .NET rekenmodel

Een proof-of-concept bij ILVO

Caradoc Roetynck

caradoc.roetynck@student.hogent.be

Promotor: Jan Willem

Co-promotor: Samuel Bosch (ILVO)

Hogeschool Gent, Valentin Vaerwyckweg 1, 9000 Gent

Samenvatting

Bij ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) ontwerpen onderzoekers complexe rekenmodellen die door IT-developers in .NET worden geïmplementeerd. Hierdoor kunnen onderzoekers hun modellen niet altijd zelf aanpassen of snel testen, wat de experimenteerruimte beperkt. Dit onderzoek toont aan dat door de integratie van Python met de bestaande C#-code, onderzoekers de autonomie krijgen om modellen direct te valideren en aan te passen, zonder de stabiliteit van de hoofdapplicatie te schaden. Naast de Python gaat dit onderzoek ook diep in Clean Code en Documentatie Drift, aangezien dit belangrijke zaken zijn om code te verduidelijken.

Keuzerichting: Software Development

Sleutelwoorden: Docker, C#, Jupyter-Notebooks, Razor

Broncode: <https://github.com/kodarakk/hogent-bachproef-caradocroetynck2526>

1. Introductie

Bij ILVO worden complexe rekenmodellen gebruikt om milieu-impact en emissies in de landbouw te berekenen. Deze modellen ontstaan vanuit wetenschappelijke kennis, maar worden in de praktijk omgezet naar software in een .NET/C#-omgeving. Daardoor ontstaat een duidelijke kloof tussen de wetenschappelijke logica en de technische implementatie.

De uitdaging

Hoewel de verdeling tussen onderzoekers en ontwikkelaars logisch is, leidt ze in de praktijk tot twee problemen:

- Beperkte zichtbaarheid: de berekeningslogica is verspreid over broncode, documentatie en overleg, waardoor onderzoekers de stappen moeilijk kunnen verifiëren.
- Afhankelijkheid: voor elke wijziging of test van een formule is technische hulp nodig, wat de experimenteerruimte beperkt en de snelheid van onderzoek vertraagt.

Het probleem is niet alleen technisch, maar ook organisatorisch: documentatie raakt snel achter op de implementatie. Daardoor ontstaat *documentation drift*, een groeiende kloof tussen wat de documentatie zegt en wat de software werkelijk doet.

Onderzoeksvraag

Hoe kunnen we de kloof tussen IT-logica en wetenschappelijke analyse verkleinen, zodat onderzoekers zonder diepgaande programmeerkennis inzicht krijgen in hun modellen en er zelf mee kunnen experimenteren?

2. Methodologie & Technologie

Het onderzoek is opgebouwd in een iteratief proces: gesprekken met onderzoekers en IT-medewerkers, literatuuronderzoek en een praktische proof-of-concept. De verzamelde inzichten worden steeds teruggekoppeld om de oplossing te verfijnen.

Gekozen aanpak:

- Probleemanalyse: gesprekken met onderzoekers en IT-medewerkers om de huidige werkwijze, knelpunten en behoeften in kaart te brengen.
- Literatuur- en technologieverkenning: onderzoek naar Python.NET, Jupyter, literate programming en documentatie-praktijken die de kloof tussen model en code kunnen verkleinen.
- Ontwerp van een hybride oplossing: een proof-of-concept waarbij C#-logica via Python beschikbaar wordt gemaakt in een veilige, interactieve analyseomgeving.
- Iteratieve evaluatie: feedback uit de praktijk gebruiken om de oplossing te verfijnen en te toetsen op bruikbaarheid, veiligheid en wetenschappelijke validatie.

Deze aanpak sluit aan bij de kern van het onderzoek: niet alleen een technische integratie, maar ook een methodiek om documentatie, experimenteren en samenwerking tussen onderzoekers en ontwikkelaars beter te laten samenwerken.

3. Clean Code

Voor ILVO is duidelijke, leesbare code niet alleen een technische keuze, maar een organisatorische noodzaak. Wanneer onderzoekers en ontwikkelaars samen werken aan complexe modellen, is het belangrijk dat de bedoeling van berekeningen, aannames en randgevallen niet alleen in een wiki of e-mail staan, maar ook in de code zelf. Goede naamgeving, kleine eenheden en duidelijke verantwoordelijkheden maken de broncode beter begrijpelijk en verminderen de afhankelijkheid van individuele kennis.

4. Documentation Drift

De onderzoeker-ervaring bij ILVO laat zien dat documentatie snel achterloopt zodra code, data en workflows niet meer synchroon lopen. Deze *documentation drift* vergroot de kans op misinterpretatie, vertraagt validatie en maakt het moeilijk om wijzigingen correct te beoordelen. Daarom is een duidelijke, centrale en versioned bron van waarheid essentieel: code, tests, notities en modelafspraken moeten beter samen evolueren. Zeker de 'waarom' is nodig om te documenteren, de developers kunnen wel de 'wat' beschrijven, maar de waarom weten zij niet. Dit is eerder een taak voor de onderzoekers.

5. Proof-of-Concept

De proof-of-concept toont dat bepaalde berekeningsonderdelen relatief eenvoudig kunnen worden vertaald naar Python, met name kleine, goed afgebakende formules en scenario's. Dit kan nuttig zijn om onderzoekers inzicht te geven in de logica en om specifieke onderdelen makkelijker te analyseren. Maar de haalbaarheid verdwijnt snel zodra het gaat om de echte dataflow, integratietests en de volledige applicatiecontext van EMAV.

Praktische lessen uit de PoC:

- Kleine, zelfstandige berekeningen zijn in Python begrijpelijker voor onderzoekers, maar vormen geen volledige vervanging van de C#-applicatie.
- Integratie-tests, unit-tests en contextgevoelige rekenstappen blijven in de praktijk beter in C# omdat de volledige domeinlogica en infrastructuur daar centraal staan.
- Hoe complexer de inputdata, custom data types en database-gestuurde flows worden, hoe moeilijker het omzetten naar Python wordt.
- De echte uitdaging ligt niet in het reproduceren van één formule, maar in het veilig en consistent laten werken van de volledige rekencontext.

Deze ervaring onderstreept het belangrijkste inzicht van het onderzoek: Python kan waarde bieden als ondersteunende analyse- en validatieomgeving, maar het doel is nooit om EMAV te vervangen. De kernlogica blijft in C# staan, terwijl Python alleen beperkt kan dienen voor selectieve onderdelen, vergelijking van resultaten en onderzoekergerichte analyse.

6. Conclusies & Toekomst

De centrale vaststelling van het onderzoek is dat de bedoeling nooit was om EMAV volledig te vervangen. De huidige .NET-architectuur is te complex en te sterk ingebed in de bedrijfslogica om zonder meer te vervangen. De praktische route is een hybride aanpak: de kern van de berekeningen blijft in een stabiele C#-omgeving, terwijl onderzoekers via Python, Jupyter-notebooks en gecontroleerde experimentatieomgevingen toegang krijgen tot geselecteerde modelonderdelen en vergelijkingsscenario's.

Deze keuze vergroot de autonomie van onderzoekers zonder de stabiliteit van de hoofdapplicatie te ondermijnen. De proof-of-concept toont aan dat Python waardevol is voor kleine onderdelen, maar dat het niet voldoende toepasbaar is om grote workflows te testen. Clean Code en Documentation Drift zijn hierbij cruciaal: ze verbeteren niet alleen de leesbaarheid van de code, maar ook de samenwerking tussen onderzoekers en IT, en verminderen de kans op misinterpretatie in een complex domeinmodel.

Toekomstig onderzoek:

- Verdiepen van de samenwerking tussen onderzoekers en ontwikkelaars om de experimenteerruimte te optimaliseren.
- Verder uitwerken van de PoC met aandacht voor clean code, veilige data-isolatie, docs-as-code en het beperken van documentation drift voor ILVO.
- Onderzoeken hoe Python kan helpen specifiek bij testen.